

ML_ALL_SVM_RF_NN

Support Vector Machines, Random Forests and Neural Networks for Protein–DNA Binding-Affinity Prediction

Abstract

This document explains, from the ground up, a machine-learning part in the paper [1]. We first build intuition for the three model families (**Support Vector Machines (SVM)**, **Random Forests (RF)** and **Neural Networks (NN)**), then show how each is adapted to the three prediction tasks in the notebook (regression, binary classification and multiclass classification). We then describe the overall architecture of the script and finally walk through the execution logic step by step, quoting the code directly.

Contents

1	Conceptual Overview	2
1.1	Support Vector Machine (SVM): drawing the widest possible boundary	2
1.2	Random Forest (RF): a committee of decision trees	2
1.3	Neural Network (NN): stacked layers of tunable knobs	2
2	Task Variations & Parameter Mapping	3
2.1	The three tasks, and how the target is chosen	3
2.2	SVM parameters (SVC for classification, SVR for regression)	3
2.3	Random Forest parameters (...Classifier / ...Regressor)	4
2.4	Neural Network parameters (MLPClassifier / MLPRegressor)	4
2.5	From fixed presets to automatic tuning: the search_space	5
3	Architectural Breakdown	5
3.1	The nine blocks	6
3.2	The PART 0–9 template inside each block	6
3.3	Data flow at a glance	6
4	Step-by-Step Logical Walkthrough	8
4.1	PART 0 — Libraries & configuration	8
4.2	PART 1–2 — Ingestion and merge	8
4.3	PART 3 — The train/test split (done by sequence)	9
4.4	PART 4 — keep_last_percent: dropping warm-up frames	9
4.5	PART 5 — Block averaging into “macrostates”	9
4.6	PART 6 — Standardization (fit on TRAIN only)	10
4.7	PART 7 — Cross-validation and per-fold training	10
4.8	PART 8 — Testing the ensemble	14
4.9	PART 9 — Automatic hyperparameter optimisation	14

1 Conceptual Overview

In this paper, it mainly focus on three models - **Support Vector Machines (SVM)**, **Random Forests (RF)** and **Neural Networks (NN)**. All three are **supervised learning** methods.

In this notebook every example is a snapshot of a simulated protein–DNA complex, the features are eight physics-based energy terms (electrostatics, van der Waals forces, entropy, and so on), and the label describes how good a binder that complex is.

1.1 Support Vector Machine (SVM): drawing the widest possible boundary

A Support Vector Machine tries to draw a single line that separates the two colours, where dots on a plane, with binders in blue and non-binders in red. The twist that makes it “support *vector*” is that it does not draw just *any* separating line — it draws the line that leaves the **widest empty margin** on either side. The handful of dots that sit right on the edge of that margin are called the *support vectors*; they alone determine where the boundary goes.

Real data is rarely separable by a straight line. SVMs handle this with a **kernel**, a mathematical trick that effectively bends the paper into a higher dimension so that a curved boundary in the original space becomes a flat one in the new space. The notebook lets the kernel be “**linear**” (a straight boundary), “**rbf**” (smooth, bubble-like boundaries), “**poly**” (polynomial curves) or “**sigmoid**” (S-shaped boundaries).

SVM

Find the boundary that separates the classes with the biggest safety margin. Use a kernel to allow curved boundaries when a straight one will not do.

1.2 Random Forest (RF): a committee of decision trees

A single **decision tree** is a flow-chart of yes/no questions: “Is the electrostatic energy below –50? If yes, go left; if no, go right,” and so on, until you reach a leaf that announces a prediction. A lone tree is easy to read but tends to *overfit* — it memorises the training examples, quirks and all, and then generalises poorly.

A **Random Forest** fixes this by growing many trees (hundreds, here) and letting them vote. Two sources of randomness keep the trees from being identical: each tree is trained on a random resample of the data, and at each question only a random subset of the features is considered. Averaging many diverse, individually-imperfect trees produces a prediction that is far more stable than any single tree.

Random Forest

Grow hundreds of slightly different decision trees and average their votes. The diversity of the trees cancels out their individual mistakes.

1.3 Neural Network (NN): stacked layers of tunable knobs

A **neural network** uses the eight input features enter a layer of artificial “neurons,” each of which computes a weighted sum of its inputs and passes the result through a simple non-linear **activation function** (the notebook uses “**relu**” or “**tanh**”). The outputs of that layer feed a second layer, and so on, until a final layer emits the prediction.

Every connection has a **weight**. Training nudges all the weights in the direction that reduces the prediction error by gradient descent / back-propagation. With enough nodes and layers, a network can approximate very complicated input–output relationships. The specific network here is a *Multi-Layer Perceptron* (MLP): a plain, fully-connected feed-forward network.

Neural Network

Pass the inputs through several layers of weighted sums and non-linear squashing functions. Training automatically tunes thousands of weights so the final output matches the known labels.

2 Task Variations & Parameter Mapping

The notebook is organised as a grid: **three model families** (SVM, RF, NN) $\times$ **three prediction tasks** (Multiclass, Binary, Regression), giving nine self-contained blocks. Crucially, every block reuses the same data pipeline; what changes between blocks is (a) which *target column* is predicted, (b) which scikit-learn *estimator class* is used, and (c) which *hyperparameters* that estimator accepts. This section maps all of that onto the precise variable names in your code.

2.1 The three tasks, and how the target is chosen

The only thing that decides whether a block is regression, binary or multiclass is the value assigned to `label_col` in the configuration cell (PART 0), together with the helper constants `NUM_CLASSES` and `CLASS_NAMES`.

Task	label_col	NUM_CLASSES	What is predicted
Multiclass	"binding_type"	3	Three-way label 0/1/2, named ["No", "Weak", "Strong"]
Binary	"improving"	2	Two-way label 0/1, named ["No", "Yes"]
Regression	"bind_avg"	1	A continuous binding energy (a real number)

Table 1: The target column and class count fully determine the task. These are set once at the top of each block.

Classification vs. regression in the code. For the two classification tasks the notebook uses scikit-learn *classifier* objects (`SVC`, `RandomForestClassifier`, `MLPClassifier`) and scores them with classification metrics (accuracy, precision, recall, F1, MCC). For regression it switches to the *regressor* twins (`SVR`, `RandomForestRegressor`, `MLPRegressor`) and scores them with regression metrics (MSE, R^2 , Pearson r). Binary and multiclass share *exactly* the same code path — the model and metrics are agnostic to how many classes there are — so the only differences are `label_col`, `NUM_CLASSES` and `CLASS_NAMES`.

2.2 SVM parameters (SVC for classification, SVR for regression)

For the classification blocks the preset (hand-picked) hyperparameters live in the variables `KERNEL`, `C` and `GAMMA`:

```
KERNEL, C, GAMMA = "sigmoid", 0.46, 0.117 # SVM classification preset
```

- `KERNEL` — the shape of the decision boundary ("linear", "rbf", "poly", "sigmoid"), as discussed in Section 1.

- **C** — the *regularisation strength*, or how harshly misclassified training points are penalised. Small **C** = a wider, smoother margin that tolerates some errors (less overfitting); large **C** = a tighter margin that tries to classify every training point correctly (more overfitting).
- **GAMMA** — for curved kernels, how far the influence of a single training point reaches. Small **GAMMA** = far-reaching, smooth boundaries; large **GAMMA** = wiggly boundaries that hug individual points.

The regression block uses the regressor SVR, which needs one extra knob and therefore stores its presets in differently-named variables:

```
SVR_KERNEL, SVR_C, SVR_GAMMA, SVR_EPSILON = "rbf", 1.81, 0.004, 0.0
```

SVR_KERNEL, SVR_C and SVR_GAMMA mean the same things as above. The new parameter SVR_EPSILON defines an “ ϵ -insensitive tube”: prediction errors smaller than SVR_EPSILON are treated as zero error, which makes the regressor ignore tiny deviations and focus on the larger ones.

2.3 Random Forest parameters (...Classifier / ...Regressor)

Both RF classification and RF regression use the *same* five preset variables; only the estimator class differs (RandomForestClassifier vs. RandomForestRegressor):

```
RF_N_ESTIMATORS      = 250      # number of trees in the forest
RF_MAX_DEPTH         = 30       # max depth of each tree (None = grow until pure)
RF_MAX_FEATURES      = "sqrt"  # how many features each split may consider
RF_MIN_SAMPLES_SPLIT = 8       # min samples required to split an internal node
RF_MIN_SAMPLES_LEAF  = 3       # min samples required at a leaf
```

- RF_N_ESTIMATORS — more trees give a steadier vote (and slower training). A few hundred is typical.
- RF_MAX_DEPTH — how many yes/no questions a tree may ask before it must decide. Deeper trees capture finer patterns but overfit more easily.
- RF_MAX_FEATURES — the randomness that de-correlates the trees: at each split only this many features are eligible. “sqrt” means “the square root of the number of features.”
- RF_MIN_SAMPLES_SPLIT / RF_MIN_SAMPLES_LEAF — guard rails that stop a tree from carving the data into tiny, noise-fitting groups.

Note that the presets differ slightly from block to block (for example RF regression ships with RF_N_ESTIMATORS = 350, RF_MAX_DEPTH = 15); the variable *names* are identical, only the chosen values change.

2.4 Neural Network parameters (MLPClassifier / MLPRegressor)

As with RF, the classifier and regressor versions of the network share one set of preset variables:

```
NN_HIDDEN_LAYER_SIZES = (64, 64) # two hidden layers of 64 neurons each
NN_ALPHA              = 1e-4      # L2 weight-decay (regularisation strength)
NN_LEARNING_RATE_INIT = 1e-3      # initial step size for gradient descent
NN_ACTIVATION         = "relu"    # non-linearity: "relu" or "tanh"
```

- NN_HIDDEN_LAYER_SIZES — the architecture. (64, 64) means two hidden layers of 64 neurons. More/bigger layers = more capacity but more risk of overfitting.

- `NN_ALPHA` — L2 regularisation: a gentle penalty on large weights that discourages the network from memorising noise.
- `NN_LEARNING_RATE_INIT` — how big a step the optimiser takes on each update. Too large and training is unstable; too small and it crawls.
- `NN_ACTIVATION` — the squashing function applied inside each neuron.

2.5 From fixed presets to automatic tuning: the `search_space`

Each block runs twice. First it trains with the fixed presets above (PARTs 0–8). Then a second pass (PART 9) lets a Bayesian optimiser (hyperopt’s TPE algorithm) *search* for better values automatically. The set of values it is allowed to try is the `search_space` dictionary. The same two preprocessing knobs (`keep_last_percent` and `navg`, explained in Section 4) are tuned in every block; the model-specific entries mirror the preset variables above:

```
# --- SVM classification search space ---
search_space = {
    "keep_last_percent": hp.quniform("keep_last_percent", 5, 100, 5),
    "navg": hp.quniform("navg", 20, 400, 20),
    "kernel": hp.choice("kernel", ["linear", "rbf", "poly", "sigmoid"]),
    "C": hp.loguniform("C", np.log(1e-3), np.log(1e3)),
    "gamma": hp.loguniform("gamma", np.log(1e-4), np.log(1.0)),
}
```

```
# --- Random Forest search space (same for both tasks) ---
search_space = {
    "keep_last_percent": hp.quniform("keep_last_percent", 5, 100, 5),
    "navg": hp.quniform("navg", 20, 400, 20),
    "n_estimators": hp.quniform("n_estimators", 50, 500, 50),
    "max_depth": hp.choice("max_depth", [10, 15, 20, 25, 30]),
    "max_features": hp.choice("max_features", ["sqrt", "log2", 0.5, 1.0]),
    "min_samples_split": hp.quniform("min_samples_split", 2, 50, 2),
    "min_samples_leaf": hp.quniform("min_samples_leaf", 1, 20, 1),
}
```

```
# --- Neural Network search space (same for both tasks) ---
search_space = {
    "keep_last_percent": hp.quniform("keep_last_percent", 5, 100, 5),
    "navg": hp.quniform("navg", 20, 400, 20),
    "hidden_layer_sizes": hp.choice("hidden_layer_sizes",
        [(50,), (100,), (50,50), (100,50), (100,100)]),
    "alpha": hp.loguniform("alpha", np.log(1e-6), np.log(1e-1)),
    "learning_rate_init": hp.loguniform("learning_rate_init",
        np.log(1e-4), np.log(1e-1)),
    "activation": hp.choice("activation", ["relu", "tanh"]),
}
```

The regression blocks add `"epsilon"` to the SVR space, and the RF regression space additionally allows `None` for `max_depth` and `max_features` (i.e. “no limit”). The three sampling helpers mean: `hp.quniform` draws stepped integers, `hp.choice` picks from a fixed menu, and `hp.loguniform` samples across many orders of magnitude (ideal for `C`, `gamma`, `alpha` and learning rates, which can span from 0.0001 to 1000).

3 Architectural Breakdown

Before reading the code line by line, it pays to see the shape of the whole thing. The notebook is long, but it is long because it *repeats one template nine times*. Understand the template once and you understand the entire notebook.

Model	Classifier / Regressor	Preset variables	Tuned keys (in search_space)
SVM	SVC (classification) SVR (regression)	KERNEL, C, GAMMA <i>regression adds:</i> SVR_KERNEL, SVR_C, SVR_GAMMA, SVR_EPSILON	kernel, C, gamma <i>regression adds:</i> epsilon
RF	RandomForestClassifier RandomForestRegressor	RF_N_ESTIMATORS RF_MAX_DEPTH RF_MAX_FEATURES RF_MIN_SAMPLES_SPLIT RF_MIN_SAMPLES_LEAF	n_estimators max_depth max_features min_samples_split min_samples_leaf
NN	MLPClassifier MLPRegressor	NN_HIDDEN_LAYER_SIZES NN_ALPHA NN_LEARNING_RATE_INIT NN_ACTIVATION	hidden_layer_sizes alpha learning_rate_init activation

Table 2: Master map from model family to its scikit-learn classes, its preset configuration variables, and the keys the optimiser tunes. Two preprocessing knobs (`keep_last_percent`, `navg`) are tuned for every model.

3.1 The nine blocks

Reading top to bottom, the notebook contains nine major sections, in this order:

1. SVM Multiclass 2. RF Multiclass 3. NN Multiclass
4. SVM Binary 5. RF Binary 6. NN Binary
7. SVM Regression 8. RF Regression 9. NN Regression

Each block is fully self-contained: it re-imports its libraries, re-reads the data, re-runs preprocessing and trains its model from scratch. That means you can run any one block in isolation, but it also means a lot of the code is duplicated. (A side-effect worth knowing: because each block re-defines shared function names like `pool_by_sequence` and re-uses variable names like `df` and `tr_std`, running blocks out of order can let one block’s definitions leak into another. Run a block top-to-bottom and you are safe.)

3.2 The PART 0–9 template inside each block

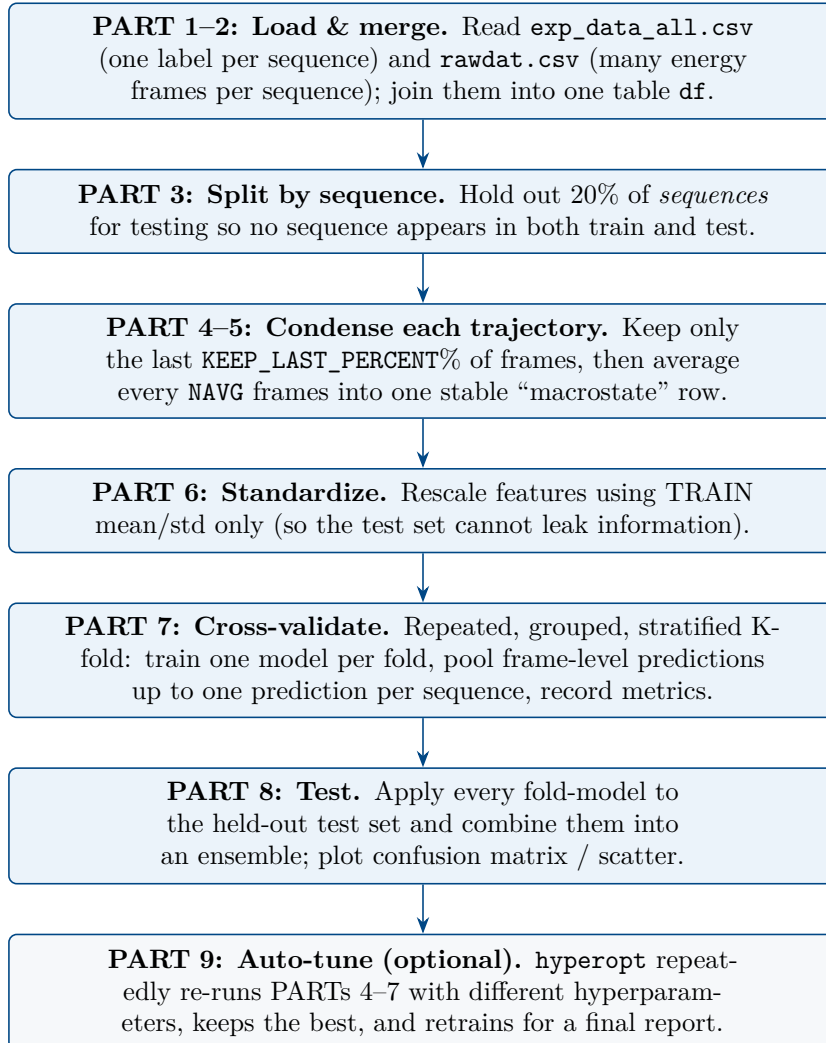
Within every block the work is divided into numbered PARTs. The headings are identical across blocks (a couple still say “train SVC” even in the RF and NN blocks — a copy-paste artefact, not a bug):

3.3 Data flow at a glance

The diagram below traces a single block from raw files to final scores. Note the two-stage structure: PARTs 0–8 use fixed presets, and PART 9 wraps the very same preprocessing-and-CV machinery inside an optimiser loop.

PART	Name	What it does
0	Libraries & config	Imports; sets <code>label_col</code> , feature list, pre-sets, seeds.
1	Data ingestion	Reads the labels CSV and the per-frame features CSV.
2	Merge (+ shuffle)	Joins features to labels; optionally shuffles.
3	Train/test split	Splits <i>by sequence</i> , stratified by class.
4	<code>keep_last_percent</code>	Drops the early “equilibration” frames of each run.
5	Block averaging	Averages groups of frames into “macrostate” rows.
6	Standardize	Centres/scales features using TRAIN statistics only.
7	K-fold CV + train	Cross-validates and trains one model per fold.
8	Test + diagnostics	Evaluates the ensemble of fold-models; plots results.
9	Bayesian optimisation	Searches for better hyperparameters, then retrains.

Table 3: The fixed PART 0–9 pipeline that every one of the nine blocks follows.



Summary

The data has a nested structure: many simulation *frames* belong to one *sequence*, and only the sequence has a true label. The entire pipeline is built to respect this — splitting, cross-validation and scoring all happen at the *sequence* level, while the model still learns from the abundant frame-level rows. Getting this right is what prevents the model from “cheating.”

4 Step-by-Step Logical Walkthrough

We now walk through one block in execution order, quoting the code directly. We use the **SVM Multiclass** block as the running example because it is the first one in the notebook; where another model or task does something different, we point it out. (Throughout, the quoted snippets are taken verbatim from the notebook.)

4.1 PART 0 — Libraries & configuration

The first cell installs and imports everything, fixes the random seed, and declares the configuration constants the rest of the block will read.

```
RANDOM_STATE = 99
np.random.seed(RANDOM_STATE)

DATA_DIR = "ML-Protein-DNA-Binding-Affinity/tutorial notebooks/Data"
id_col, run_col = "sequence", "run"
label_col = "binding_type" # multiclass label (0/1/2)
NUM_CLASSES = 3
CLASS_NAMES = ["No", "Weak", "Strong"]
feature_cols = ["VDWAALS", "EEL", "EGB", "ESURF", "HB Energy",
                "Hydrophobic Energy", "Pi-Pi Energy", "Delta_Entropy"]
```

Why this matters. Two ideas are doing a lot of work here. First, `RANDOM_STATE = 99` (and `np.random.seed`) makes the run *reproducible*: every “random” choice later — shuffling, splitting, forest building — will come out the same way each time, so results can be compared fairly. Second, the column names are named once as variables (`id_col`, `label_col`, `feature_cols`) and reused everywhere; this is what lets the same code serve all nine blocks by changing only `label_col`.

The `feature_cols` are the eight physics-based energy terms (van der Waals, electrostatics, solvation, hydrogen-bonding, hydrophobic, π - π stacking and an entropy term). These eight numbers are the only information the model ever sees about an example.

4.2 PART 1–2 — Ingestion and merge

```
ref = pd.read_csv(f"{DATA_DIR}/exp_data_all.csv")[[id_col, label_col]].copy()
md = pd.read_csv(f"{DATA_DIR}/rawdat.csv", usecols=[id_col, run_col, *feature_cols])
df = pd.merge(md, ref, on=id_col, how="inner")
```

There are two source files. `exp_data_all.csv` (here called `ref`) holds *one label per sequence* — the experimentally known answer. `rawdat.csv` (`md`) holds the *many simulation frames per sequence*, each with the eight energy features. The `pd.merge(..., on=id_col)` glues each frame to its sequence’s label, producing `df`: a long table where every row is one frame and carries both its features and its (shared) label.

```
df_chronological = df.copy()
if SHUFFLE_MERGE: # destroys frame chronology (control behavior)
    df = df.sample(frac=1.0, random_state=RANDOM_STATE).reset_index(drop=True)
df_pre_keep = df.copy()
```


Why shuffle? The simulation frames arrive in time order. The author keeps a pristine time-ordered copy (`df_chronological`) for plotting, then optionally shuffles the working copy. Shuffling is a deliberate “control”: it scrambles the time order so any later step that secretly relied on chronology would be exposed. `df_pre_keep` is yet another snapshot kept for a before/after diagnostic plot.

4.3 PART 3 — The train/test split (done by sequence)

This is the first place the nested data structure forces a careful choice.

```
seq_label = df.groupby(id_col)[label_col].first()          # one class per sequence
train_seqs, test_seqs = train_test_split(
    seq_label.index.to_numpy(), test_size=0.20,
    random_state=RANDOM_STATE, stratify=seq_label.to_numpy())
df_train = df[df[id_col].isin(set(train_seqs))].copy()
df_test  = df[df[id_col].isin(set(test_seqs))].copy()
...
assert not (set(train_seqs) & set(test_seqs)), "LEAKAGE!"
```

Why split sequences, not rows? If we split the individual frame-rows at random, frames from the *same* sequence would land in both the training and test sets. Since those frames look almost identical and share a label, the model could effectively “study the answer key”: it would score well on the test set without having truly generalised. This is called *data leakage*. Splitting whole sequences (the `id_col` groups) guarantees a test sequence is genuinely unseen. The explicit `assert ... "LEAKAGE!"` is a safety check that crashes loudly if any sequence ended up on both sides.

Why stratify? Passing `stratify=seq_label` asks the splitter to preserve the proportion of each class (No/Weak/Strong) in both the training and test sets. Without it, a small class could vanish entirely from one side by chance.

4.4 PART 4 — keep_last_percent: dropping warm-up frames

A molecular-dynamics simulation needs time to “settle” from its artificial starting point. Those early, unphysical frames are noise. `keep_last_n_percent` discards them, keeping only the trailing, equilibrated portion of each run:

```
sizes      = df_sorted.groupby(group_cols)[seq_col].transform("size")
position   = df_sorted.groupby(group_cols).cumcount()
n_keep     = (sizes * (keep_percent / 100.0)).astype(int).clip(lower=1)
mask       = position >= (sizes - n_keep)
return df_sorted[mask].reset_index(drop=True)
```

For each group it computes the group size, numbers the rows 0, 1, 2, ... (`cumcount`), works out how many trailing rows to keep, and keeps only rows whose position is in that final window. The call

```
df_train = keep_last_n_percent(df_train, id_col, KEEP_LAST_PERCENT, per_run=True)
```

uses `per_run=True`, so the “last X%” is computed *within each replica run* rather than across a sequence’s runs lumped together — the correct unit, since each run equilibrates on its own. `KEEP_LAST_PERCENT` is one of the two preprocessing knobs the optimiser later tunes. The accompanying `plot_trajectory_lifecycle` helper overlays the chronological, pre-keep and post-keep versions of one trajectory so you can visually confirm the right frames were kept.

4.5 PART 5 — Block averaging into “macrostates”

Individual frames are noisy. Averaging consecutive frames into a single row smooths that noise and produces more reliable training examples, which the author calls *macrostates*.

```
def block_average(d, navg):
    recs = []
    for _, g in d.groupby(id_col):
        block = min(navg, len(g))
        feats = [c for c in g.select_dtypes(include=[np.number]).columns
                  if c not in {id_col, label_col, run_col}]
        sh = g.sample(frac=1.0, random_state=RANDOM_STATE).reset_index(drop=True)
        for i in range(len(sh)//block):
            ch = sh.iloc[i*block:(i+1)*block]
            r = {c: ch[c].mean() for c in feats}
            r[label_col] = ch[label_col].iloc[0]; r[id_col] = ch[id_col].iloc[0]
            recs.append(r)
    ...
```

For each sequence it shuffles the kept frames, slices them into chunks of `navg` frames, and replaces each chunk with the *mean* of its features. The chunk’s label and sequence id are carried over unchanged (all frames of a sequence share them). The result `tr_avg` has far fewer rows than the raw frame table, but each row is a steadier summary. `NAVG` is the second preprocessing knob the optimiser tunes: large `NAVG` = fewer, smoother rows; small `NAVG` = more, noisier rows.

4.6 PART 6 — Standardization (fit on TRAIN only)

The eight energy features live on wildly different numeric scales. SVMs and neural networks are sensitive to that, so each feature is rescaled to roughly zero mean and unit spread:

```
feat = [c for c in tr_avg.columns if c not in (id_col, label_col)]
mu, sd = tr_avg[feat].mean(), tr_avg[feat].std(ddof=1)
tr_std = tr_avg.copy(); tr_std[feat] = (tr_avg[feat] - mu) / sd.replace(0, 1)
te_std = te_avg.copy(); te_std[feat] = (te_avg[feat] - mu) / sd.replace(0, 1)
```

The subtle, important detail: the mean `mu` and standard deviation `sd` are computed from the *training* data only, and then applied to *both* train and test. If the test set’s own statistics were used, a sliver of test information would leak into preprocessing. The `sd.replace(0, 1)` guards against dividing by zero for a constant feature. A small helper packages a table into model-ready arrays:

```
def to_xy(d):
    return d[feat].values.astype(np.float32), d[label_col].values.astype(int), d[id_col].tolist()
```

It returns `x` (the feature matrix), `y` (the labels) and the list of sequence ids — the ids are kept precisely so predictions can later be pooled back to the sequence level. In the *regression* blocks this helper is named `make_xy` and casts `y` to a float instead of an int, because the target is a continuous number rather than a category.

4.7 PART 7 — Cross-validation and per-fold training

This is the heart of the block. Three nested ideas combine: repeated runs, grouped+stratified K-fold splitting, and pooling predictions to the sequence level.

The splitter.

```
for repeat_index in range(NUM_REPEATS):
    skf = StratifiedGroupKFold(
        n_splits=KFOLD, shuffle=True,
        random_state=RANDOM_STATE + repeat_index)
    for fold_index, (train_indices, val_indices) in enumerate(
        skf.split(tr_std, all_training_labels, sequence_group_ids)):
```

StratifiedGroupKFold does two jobs at once: *Group* keeps all rows of one sequence inside the same fold (no leakage, exactly as in PART 3), and *Stratified* keeps the class proportions balanced across folds. Wrapping it in a NUM_REPEATS loop (with a different seed each time) repeats the whole K-fold procedure several times, which averages out the luck of any single split. With NUM_REPEATS=3 and KFOLD=5 this trains $3 \times 5 = 15$ models.

The splitter for regression. The regression blocks cannot use StratifiedGroupKFold, because “stratifying” requires discrete classes and the regression target is a continuous number. Instead the author writes a small custom splitter, `make_regression_folds`, that keeps the *group* guarantee (a whole sequence stays in one fold) but drops stratification:

```
def make_regression_folds(data, n_splits, seed):
    """Split whole SEQUENCES into K folds (a sequence's rows always stay together)."""
    seqs = np.array(sorted(data[id_col].unique()))
    rng = np.random.default_rng(seed); rng.shuffle(seqs)
    group_ids = data[id_col].values
    for fold_seqs in np.array_split(seqs, n_splits):
        is_val = np.isin(group_ids, fold_seqs)
        yield np.where(~is_val)[0], np.where(is_val)[0]
```

It collects the unique sequence ids, shuffles them, and splits that list of *sequences* (not rows) into `n_splits` groups with `np.array_split`. For each group it marks all rows whose sequence is in that group as the validation fold (`is_val`) and the rest as training, then yields the two index arrays — exactly the (`train_idx`, `val_idx`) pairs that the training loop expects. The regression training loop therefore looks almost identical to the classification one, only with this splitter and an SVR / ...Regressor model in place of the classifier:

```
for repeat_index in range(NUM_REPEATS):
    for fold_index, (train_idx, val_idx) in enumerate(
        make_regression_folds(tr_std, KFOLD, RANDOM_STATE + repeat_index)):
        X_train, y_train, _ = make_xy(tr_std.iloc[train_idx])
        X_val, y_val, val_ids = make_xy(tr_std.iloc[val_idx])
        model = SVR(kernel=SVR_KERNEL, C=SVR_C, gamma=SVR_GAMMA, epsilon=SVR_EPSILON)
        model.fit(X_train, y_train)
        trained_models.append(model)
        true_seq, pred_seq = pool_by_sequence(val_ids, model.predict(X_val), y_val)
        cv_rows.append(regression_metrics(true_seq, pred_seq))
```

Training one fold.

```
X_train, y_train, _ = to_xy(df_fold_train)
X_val, y_val, val_sequence_ids = to_xy(df_fold_val)

model = SVC(kernel=KERNEL, C=C, gamma=GAMMA, max_iter=1_000_000,
            cache_size=1000, random_state=RANDOM_STATE, probability=True)
model.fit(X_train, y_train)
trained_rf_models.append(model)
```

Each fold trains a fresh model on its training rows and is stored in the list `trained_rf_models` (the name is a leftover from the RF block; in the SVM block it holds SVCs). The RF and NN blocks differ here only in the constructor line — `RandomForestClassifier(...)` or `MLPClassifier(...)` — while everything around it is identical.

From frames to a sequence verdict. For classification, the model predicts a label for every macrostate row, but we ultimately care about one verdict per *sequence*. To predict, for the case of binary, it would be simply using `model.predict(X)` to perform classification. For multiclass, we introduce `predict_proba_full`.

```
def predict_proba_full(model, X):
    proba = model.decision_function(X)      # SVM: geometric margins
    full = np.zeros((X.shape[0], NUM_CLASSES))
    for col, cls in enumerate(model.classes_):
        full[:, int(cls)] = proba[:, col]
    return full
```

`predict_proba_full` produces a score for every one of the `NUM_CLASSES` classes, padding with zeros for any class a particular fold never saw in training (otherwise the column layout would shift). In the SVM block this reads the geometric `decision_function`; in the RF and NN blocks the identical-looking helper instead calls `model.predict_proba(X)`, since those models output genuine probabilities.

```
def pool_by_sequence(seq_ids, soft_predictions, true_labels):
    bucket = defaultdict(lambda: {"proba": [], "true": []})
    for sid, pr, yt in zip(seq_ids, soft_predictions, true_labels):
        bucket[sid]["proba"].append(pr); bucket[sid]["true"].append(int(yt))
    true = np.array([get_majority_vote(b["true"]) for b in bucket.values()])
    pred = np.array([int(np.argmax(np.sum(b["proba"], axis=0)))
                     for b in bucket.values()])
    return true, pred
```

`pool_by_sequence` groups all macrostate predictions belonging to one sequence, *sums* their class scores and takes the `argmax` (the highest-scoring class) as the sequence's single prediction; the sequence's true label is the **majority vote** of its chunk labels.

Detail step after prediction: What goes in, what comes out (a worked example). The function receives three parallel lists — one entry per macrostate *chunk*: the sequence id it belongs to (`seq_ids`), the model's per-chunk prediction (`soft_predictions`), and the chunk's true label (`true_labels`). It returns two short arrays — one entry per *sequence*.

Step 1: one sequence (A) split into three chunks. Suppose after chunking, sequence A produced three macrostates, and the model predicted the labels [1, 1, 0] for them while their true labels were [1, 1, 1]:

```
# INPUT (one row per chunk)
seq_ids      = ["A", "A", "A"]
model prediction = [ 1, 1, 0 ]    -> majority vote = 1    (two 1s beat one 0)
true labels   = [ 1, 1, 1 ]    -> majority vote = 1

# OUTPUT (one row per sequence)
pred_seq = [1]
true_seq = [1]
```

Step 2: three distinct sequences (A, B, C). After the within sequence pooling, since there are five folds in each round (training five models in each round. If `NUM_REPEATS` = 3, we have total 15 models), we would have five pooled prediction for each sequence in each round. The output for each fold has one verdict per sequence:

```
# OUTPUT
pred_seq = [1, 0, 1]
true_seq = [1, 0, 1]
```

Hard vs. soft class-scores. The examples above used *hard* predictions. For the case of binary, the code pools the *hard* ones. For the case of multiclass, the code pools *soft* ones — the distinction is written below.

- A **hard** class-score is the single discrete label the model commits to for a chunk: just the class number 0/1/2. A plain majority vote works on these.

- A **soft** class-score is the model's graded confidence *before* it commits — a probability per class (Random Forest and the neural net, via `predict_proba`) or a signed distance from the decision boundary (SVM, via `decision_function`). It carries more information than the hard label because it also says *how sure* the model is.

For sequence A's three chunks, the two views look like this:

```
# HARD prediction: one committed label per chunk (what a plain vote uses)
seq_ids      = ["A", "A", "A"]
model prediction = [ 1 ,  1 ,  0 ]      -> majority vote   = 1

# SOFT prediction: a confidence score per chunk (what pool_by_sequence uses)
seq_ids      = ["A", "A", "A"]
model prediction = [ 0.6, 0.4, 0.0 ]    -> sum of scores -> argmax = 1
```

In the multiclass, `pool_by_sequence` keeps the soft scores, *sums* them across a sequence's chunks, and takes the `argmax` (the highest-scoring class) as the sequence's prediction. (In the real multiclass code each chunk's soft score is actually a full vector with one entry per class, e.g. `[0.6, 0.4, 0.0]` for `[No, Weak, Strong]`; the vectors are summed column-by-column and then a single `argmax` is taken.) For confident, agreeing chunks this gives the same answer as a plain majority vote — but soft pooling lets one *very* confident chunk outweigh several lukewarm ones, which a hard vote cannot do. The true label, by contrast, uses an explicit majority vote via `get_majority_vote`, which is just `pd.Series(labels).mode()`.

The key point is that the abundant, noisy *chunk-level* predictions are collapsed into one clean *sequence-level* verdict that can be compared against the sequence's known answer.

For regression the helper is rewritten — there are no classes to vote on, so it simply *averages* the predicted numbers per sequence:

```
def pool_by_sequence(seq_ids, predicted_values, true_values):
    """One value per sequence = MEAN of its macrostate values."""
    bucket = defaultdict(lambda: {"pred": [], "true": []})
    for sid, yp, yt in zip(seq_ids, predicted_values, true_values):
        bucket[sid]["pred"].append(float(yp)); bucket[sid]["true"].append(float(yt))
    true = np.array([np.mean(b["true"]) for b in bucket.values()])
    pred = np.array([np.mean(b["pred"]) for b in bucket.values()])
    return true, pred
```

So for a regression sequence whose three chunks were predicted `[-0.4, -0.6, -0.2]`, the pooled prediction is their mean, `-0.4`.

Scoring.

```
def classification_metrics(true_seq, pred_seq):
    if len(set(true_seq.tolist())) <= 1:
        return {"ACCURACY": nan, "PRECISION": nan, ...} # skip single-class folds
    return {"ACCURACY": accuracy_score(...), "PRECISION": precision_score(..., average="macro"),
            "RECALL": ..., "F1": ..., "MCC": matthews_corrcoef(...)}
```

Five metrics are recorded. *Accuracy* is the fraction correct; *macro precision/recall/F1* average the per-class scores so a rare class counts as much as a common one; *MCC* (Matthews Correlation Coefficient) is a balanced score that stays honest even when classes are imbalanced. The guard returns NaN for any fold that accidentally contains only one class, and because pandas' `.mean()` skips NaNs, such folds are simply ignored when averaging. **The regression blocks** swap this for `regression_metrics`, reporting MSE, R^2 and Pearson r — and, as a bonus, they threshold the predicted number at `REGRESSION_THRESHOLD = 0.0` to also report a classification-style accuracy and MCC.

4.8 PART 8 — Testing the ensemble

PART 7 left us with 15 trained fold-models. PART 8 turns them into one prediction each and then combines them:

```
for fold_index, rf_model in enumerate(trained_rf_models):
    true_labels_test, predicted_labels_test = pool_by_sequence(
        test_sequence_ids, predict_proba_full(rf_model, X_test), y_test)
    test_metrics_list.append(classification_metrics(true_labels_test, predicted_labels_test))
    ensemble_votes.append(predicted_labels_test)

ensemble_votes = np.array(ensemble_votes)          # (n_models, n_sequences)
final_pred = [int(pd.Series(ensemble_votes[:, j]).mode()[0]) # majority vote
               for j in range(ensemble_votes.shape[1])]
```

Two levels of voting. From step 7, after the within sequence pooling, since there are five folds in each round (training five models in each round. If $NUM_REPEATS = 3$, we have total 15 models), we would have five pooled prediction for each sequence in each round. To get the final prediction for each sequence, we would pool again across those five results (If $NUM_REPEATS = 3$, pool 15 results) so the output has one verdict per sequence. More specifically, there are now *two* separate aggregation steps stacked on top of each other:

1. **Inside one fold-model** (the loop body): that single model predicts every test chunk, and `pool_by_sequence` collapses those chunks into one prediction per sequence — the first vote, *within* a model. This gives `predicted_labels_test`, which is appended to `ensemble_votes`.
2. **Across the 15 fold-models** (after the loop): `ensemble_votes` is now a grid of shape $(n_models, n_sequences)$. For each sequence (each column `j`) the code takes `.mode()` — the most common label among the 15 models — to produce `final_pred`. This is the second vote, *across* models.

So a sequence’s final label is “the answer the majority of the 15 fold-models arrived at,” where each model had already condensed its own chunk-level guesses into a single per-sequence answer. Using the committee of 15 rather than any one model is more robust — the same wisdom-of-the-crowd logic that makes a Random Forest work, applied one level up. (The per-model metric rows are also averaged separately, giving the “avg of folds” line you see in the summary table, next to the “ensemble” line that scores `final_pred`.)

Reading the confusion matrix. The results are then drawn as a *confusion matrix*: a square grid with the true class along the rows and the predicted class along the columns, so cell (i, j) counts how many sequences whose true class was i were predicted as class j . A perfect model lights up only the diagonal (true = predicted); every off-diagonal cell is a specific kind of mistake (e.g. “Weak binders called Strong”). The notebook colours the diagonal green and the off-diagonal cells red, and annotates each cell with both the raw count and the row-percentage, so you can see at a glance not just *how often* the model is right but *which classes it confuses*. Two matrices are shown side by side — one for the cross-validation predictions, one for the held-out test set.

In the regression blocks there is no voting and no confusion matrix: the ensemble instead *averages* the 15 fold-models’ numeric predictions, and the plot becomes a predicted-vs-true scatter with the ideal $y = x$ line drawn through it (points near the line = accurate predictions).

4.9 PART 9 — Automatic hyperparameter optimisation

Everything so far used the hand-picked presets from PART 0. PART 9 asks: can the computer find *better* settings on its own?

The big picture: what hyperparameter optimisation is. A model has two kinds of numbers. The *parameters* (an SVM’s support vectors, a tree’s split thresholds, a network’s weights) are learned automatically during fit. The *hyperparameters* (`C`, `gamma`, `max_depth`, the layer sizes, and the two preprocessing knobs) are *not* learned — you must choose them before training. Different choices can change the result dramatically, and there are far too many combinations to try by hand. Hyperparameter optimisation automates that search: propose a combination, measure how good it is, and use what you have learned to propose a better one.

How TPE works (intuition). The simplest searches are *grid search* (try every combination on a grid) and *random search* (try random combinations); both are “memoryless” — they never learn from the trials already run. `hyperopt` instead uses **TPE**, the *Tree-structured Parzen Estimator*, a form of *Bayesian optimisation*. The idea is to build a probabilistic model of the relationship between hyperparameters and the resulting loss, and let that model guide where to look next:

1. Run a few trials to get started (it samples the space at first).
2. Sort the trials seen so far into a “good” group (low loss) and a “bad” group (high loss), split at a quantile of the observed losses.
3. Fit two probability densities over the hyperparameters: $\ell(x)$ from the good group and $g(x)$ from the bad group (these are the “Parzen estimators” — smoothed histograms over the values that did well vs. poorly).
4. Pick the next combination to try where the ratio $\ell(x)/g(x)$ is highest — i.e. values that look much more typical of *good* trials than of *bad* ones. This automatically balances *exploiting* promising regions with *exploring* uncertain ones.

Because each new trial sharpens ℓ and g , the search keeps narrowing toward good settings instead of guessing blindly — usually finding a strong combination in far fewer trials than grid or random search. (“Tree-structured” refers to how it handles conditional spaces, where some hyperparameters only matter given the value of another.)

What you must define to use hyperopt. Using `hyperopt` always comes down to three ingredients, which map directly onto the code in this part:

1. **A search space** — the menu of values each hyperparameter may take. This is the `search_space` dictionary from Section 2 (built with `hp.quniform`, `hp.choice`, `hp.loguniform`).
2. **An objective function** — a Python function that accepts one candidate `params`, trains/evaluates with it, and returns `{"loss": <number>, "status": STATUS_OK}`. The optimiser only ever sees this single number; *smaller is better*.
3. **The driver** — a call to `fmin(...)` that ties the objective, the space and the `tpe.suggest` algorithm together and runs the search for `max_evals` trials, recording everything in a `Trials()` object.

The objective function for the SVM classification block is below.

```
def SVM_cv_loss(params):
    keep = int(params["keep_last_percent"]); navg = max(20, int(params["navg"]))
    tr_std_t, _, feat_t = preprocess(raw_train_frames, None, keep, navg)
    ...
    for rep in range(OPT_NUM_REPEATS):
        sgkf = StratifiedGroupKFold(n_splits=OPT_KFOLD, shuffle=True, ...)
        for tr_idx, va_idx in sgkf.split(tr_std_t, y, groups):
            m = SVC(kernel=params["kernel"], C=float(params["C"]), gamma=float(params["gamma"]
            ], ...)
```

```

m.fit(Xtr, ytr)
t, p = pool_by_sequence(va_id, predict_proba_full(m, Xva), yva)
accs.append(accuracy_score(t, p)); mcs.append(matthews_corrcoef(t, p))
loss = W_ACC * (1 - np.mean(accs)) + W_MCC * (1 - np.mean(mcs))
return {"loss": loss, "status": STATUS_OK}

```

The *objective function* takes one candidate set of `params`, runs the *entire* PART 4–7 pipeline on the training data only (via the `preprocess` helper, which re-bundles `keep`→`average`→`standardize` so each trial can use a different `keep_last_percent` and `navg`), and returns a single `loss`. Critically, **the test set is never touched inside the search** — only training data is used — so the final test score remains an honest estimate.

The loss function being minimised. The single number returned is what TPE is trying to drive down, so its exact definition matters. The two tasks use different but parallel blends, each weighting its two components equally (0.5/0.5, from `W_ACC`, `W_MCC` = 0.5, 0.5). Because the search *minimises*, each “higher-is-better” metric is turned into a “lower-is-better” term by subtracting it from 1:

- **Classification** — minimises a 50/50 blend of $(1 - \text{accuracy})$ and $(1 - \text{MCC})$:

$$\text{loss} = 0.5 (1 - \overline{\text{accuracy}}) + 0.5 (1 - \overline{\text{MCC}}),$$

where the bars denote the mean over all CV folds. A perfect classifier ($\text{accuracy} = \text{MCC} = 1$) gives loss 0.

- **Regression** — minimises a 50/50 blend of MSE and $(1 - \text{Pearson } r)$:

$$\text{loss} = 0.5 \overline{\text{MSE}} + 0.5 (1 - \overline{\text{Pearson } r}).$$

MSE is already “lower-is-better,” while the correlation r (higher-is-better) is flipped to $1 - r$. A perfect regressor ($\text{MSE} = 0$, $r = 1$) gives loss 0.

Pairing a threshold-based score (accuracy/MCC, or MSE) with a rank/shape-based score (MCC, or Pearson r) rewards combinations that are both *accurate* and *well-correlated*, rather than gaming a single metric.

```

trials = Trials()
best_raw = fmin(fn=SVM_cv_loss, space=search_space, algo=tpe.suggest,
               max_evals=N_TRIALS, trials=trials,
               rstate=np.random.default_rng(RANDOM_STATE))
best_params = space_eval(search_space, best_raw)

```

`fmin` runs `N_TRIALS` (here 50) candidate evaluations and returns the best. `space_eval` converts `hyperopt`’s internal integer codes back into the human-readable values (e.g. turning a kernel index of 3 back into “sigmoid”). Finally, `train_and_evaluate(best_params)` re-runs PARTs 7–8 one last time with the winning settings and prints the final report. The regression block follows the very same three-ingredient recipe — search space, objective, `fmin` — differing only in the regression loss given above and in using `SVR` / ...`Regressor` models.

References

- [1] Carmen Al Masri and Jin Yu, “Combining Physics-Based Protein–DNA Energetics with Machine Learning to Predict Interpretable Transcription Factor–DNA Binding,” *J. Chem. Inf. Model.* **2025**, *65*, 11804–11817.